

CORPORACIÓN UNIVERSITARIA DEL HUILA

CORHUILA

Facultad de Ingeniería — Programa de Ingeniería de Sistemas

Taller: Defect Density

Integrantes:

Anderson Kaleth Camacho Cabezas

David Felipe Perdomo Castillo

Elkin Stiven Contreras Rojas

Edwar Alexis Serrudo Ariza

David Santiago Gomez Gutierrez

Docente:

Jose Miguel Llanos Mosquera

Asignatura:

Calidad de Software

Neiva, Huila — 2025

Tabla de Contenido

1. Introducción	3
2. Desarrollo	4
2.1. Identificación de Issues en SonarQube	4
2.2. Cálculo de Defect Density	5
2.3. Descripción y 2.4. Solución de cada Issue	6
Issue 1	—
Issue 2	—
Issue 3	—
Issue 4	—
Issue 5	—
Issue 6	—
Issue 7	—
Issue 8	—
Issue 9	—
Issue 10	—
Issue 11	—
Issue 12	—
Issue 13	—
Issue 14	—
Issue 15	—
Issue 16	—
Issue 17	—
Issue 18	—
Issue 19	—
Issue 20	—
Issue 21	—
Issue 22	—
2.5. Implementación de los Cambios	—
2.6. Comparación Antes y Después en SonarQube	—
3. Conclusiones	—
Referencias	—

1. Introducción

La densidad de defectos (Defect Density) es una métrica fundamental en ingeniería de software que permite cuantificar la calidad de un producto midiendo la cantidad de defectos por cada mil líneas de código (KLOC). Su análisis facilita la identificación temprana de problemas estructurales, orienta los esfuerzos de refactorización y permite comparar la evolución de la calidad del software a lo largo del tiempo.

El presente taller aplica esta métrica sobre el proyecto *defect-density*, desarrollado en Java, utilizando IntelliJ IDEA como entorno de desarrollo y SonarQube como herramienta de análisis estático de código. Se identificaron 22 issues clasificados por severidad (High, Medium y Low), se calculó la densidad de defectos respecto a la meta estándar de menos de 5 defectos/KLOC, y se implementaron soluciones basadas en buenas prácticas, código limpio y patrones de diseño reconocidos.

Finalmente, se ejecutó un segundo análisis en SonarQube para comparar los resultados antes y después de las correcciones, confirmando la eliminación total de los 22 issues detectados inicialmente, con una Defect Density final de 0 defectos/KLOC.

2. Desarrollo

2.1. Identificación de Issues en SonarQube

Tras configurar el proyecto *defect-density* en SonarQube y ejecutar el análisis estático, se identificaron un total de 22 issues distribuidos de la siguiente manera:

Severidad	Cantidad
High	2
Medium	13
Low	7
Total	22

[Insertar captura de pantalla del dashboard de SonarQube mostrando los 22 issues.]

2.2. Cálculo de Defect Density

La fórmula estándar para calcular la densidad de defectos es:

$$\text{Defect Density} = (\text{Número de defectos} / \text{Líneas de código}) \times 1000$$

Parámetro	Valor
Líneas de código (LOC)	73
Número de defectos	22
Defect Density	$(22 / 73) \times 1000 = 301.36$ defectos/KLOC
Meta de referencia	< 5 defectos/KLOC

Interpretación: Un Defect Density de 301.36 defectos/KLOC es extremadamente alto frente a la meta de menos de 5 defectos/KLOC. Esto indica que el código tiene problemas frecuentes casi cada pocas líneas e implica:

Alta probabilidad de fallos en ejecución y comportamientos inesperados.

Código difícil de mantener y escalar.

Cada cambio puede introducir nuevos errores (efecto bola de nieve).

2.3. Descripción y 2.4. Solución de cada Issue

A continuación se presenta cada issue identificado por SonarQube, incluyendo su descripción, antipatrón asociado y la solución implementada con código fuente.

2.3.1. Issue 1 — Alta Complejidad Cognitiva en processUser()

Severidad: High

El método processUser presenta una complejidad cognitiva de nivel 18, superando el límite permitido de 15 según SonarQube. Esto se debe a múltiples estructuras if anidadas que dificultan la lectura, comprensión y mantenimiento del código, rompiendo el flujo lineal y aumentando el esfuerzo mental para seguir la lógica.

Antipatrón: Deep Nesting / High Cognitive Complexity

2.4.1. Solución Issue 1 — Alta Complejidad Cognitiva en processUser()

Se refactorizó el método aplicando guard clauses e isEmpty(), combinando condiciones con && para eliminar el anidamiento excesivo y reducir la complejidad cognitiva por debajo de 15.

Código con el problema:

```
public String processUser(String user) {  
    if (user != null) {  
        if (user.length() > 0) {  
            if (user.contains("admin")) {  
                if (user.contains("test")) {  
                    if (user.contains("prod")) {  
                        return "ADMIN_TEST_PROD";  
                    } else { return "ADMIN_TEST"; }  
                } else { return "ADMIN"; }  
            }  
        }  
    }  
    return "UNKNOWN";  
}
```

Código corregido:

```
public String processUser(String user) {  
    if (user == null || user.isEmpty()) return "UNKNOWN";
```

```

if (user.contains("admin") && user.contains("test") && user.contains("prod"))
return "ADMIN_TEST_PROD";
if (user.contains("admin") && user.contains("test"))
return "ADMIN_TEST";
if (user.contains("admin")) return "ADMIN";
return "UNKNOWN";
}

```

2.3.2. Issue 2 — *Fuga de Recursos en Files.lines()*

Severidad: High

El Stream creado con Files.lines() no se cierra después de su uso, provocando una fuga de recursos que puede aumentar el consumo de memoria y afectar el rendimiento. Si ocurre una excepción, el recurso queda abierto indefinidamente.

Antipatrón: Resource Leak

2.4.2. Solución Issue 2 — *Fuga de Recursos en Files.lines()*

Se implementó try-with-resources para garantizar el cierre automático del Stream. Adicionalmente se reemplazó System.out por un Logger y se especificó IOException en lugar de la excepción genérica.

Código con el problema:

```

public String readUserFile(String path) throws Exception {
Stream<String> lines = Files.lines(Paths.get(path));
lines.forEach(System.out::println);
return "File read";
}

```

Código corregido:

```

public String readUserFile(String path) throws IOException {
try (Stream<String> lines = Files.lines(Paths.get(path))) {
lines.forEach(logger::info);
}
return "File read";
}

```

2.3.3. Issue 3 — *Lógica Incorrecta en deposit() y uso de System.out*

Severidad: Medium

El método deposit detecta un monto negativo pero igualmente lo suma al balance, lo cual es incoherente. Además se usa System.out.println en lugar de un logger para el manejo de registros. En isSameAccount se comparan Strings con == en lugar de equals().

Antipatrón: Logic Error + Inappropriate Logging

2.4.3. Solución Issue 3 — Lógica Incorrecta en deposit() y uso de System.out

Se corrigió la lógica con un return temprano (o excepción) para evitar sumar montos negativos. Se eliminó System.out y se usó IllegalArgumentException para el manejo correcto del error.

Código con el problema:

```
public void deposit(double amount) {  
    if (amount < 0) System.out.println("Error");  
    balance += amount; // suma igual aunque sea negativo  
}
```

Código corregido:

```
public void deposit(double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("El monto debe ser positivo");  
    }  
    balance += amount;  
}
```

2.3.4. Issue 4 — Comparación con == en Strings y Código Muerto en getBalance()

Severidad: Medium

En isSameAccount se usa == para comparar Strings, lo que compara referencias en memoria en lugar del contenido, generando resultados incorrectos. En getBalance la variable temp recibe un valor que nunca se utiliza, constituyendo código muerto.

Antipatrón: Identity vs Equality + Dead Code

2.4.4. Solución Issue 4 — Comparación con == en Strings y Código Muerto en getBalance()

Se reemplazó == por equals() para comparar Strings por contenido. Se eliminó la variable temp en getBalance() que almacenaba un cálculo sin uso.

Código con el problema:

```
public boolean isSameAccount(BankAccount other) {  
    return this.accountNumber == other.accountNumber; // referencia, no contenido  
}  
  
public double getBalance() {  
    double temp = balance * 2; // nunca usada  
    return balance;  
}
```

Código corregido:

```
public boolean isSameAccount(BankAccount other) {  
    return this.accountNumber != null  
    && this.accountNumber.equals(other.accountNumber);  
}  
  
public double getBalance() {  
    return balance;  
}
```

2.3.5. Issue 5 — Comparación con == y Variable No Utilizada

Severidad: Medium

Mismos problemas que el Issue 4: comparación incorrecta con == en isSameAccount para objetos String, y variable temp asignada pero nunca utilizada en getBalance(), ambos afectando la corrección y legibilidad del código.

Antipatrón: Identity vs Equality + Dead Code

2.4.5. Solución Issue 5 — Comparación con == y Variable No Utilizada

Se aplica la misma corrección: equals() para comparar Strings y eliminación de la variable temp sin uso en getBalance().

Código con el problema:

```
public boolean isSameAccount(BankAccount other) {  
    return this.accountNumber == other.accountNumber;  
}
```

Código corregido:

```
public boolean isSameAccount(BankAccount other) {
```



```

return this.accountNumber != null
&& this.accountNumber.equals(other.accountNumber);
}
public double getBalance() {
return balance;
}

```

2.3.6. Issue 6 — *Uso Inadecuado de printf y System.out*

Severidad: Medium

Se emplea printf sin especificadores de formato, lo que resulta innecesario y confuso. Además se usa System.out para registros, lo cual no es adecuado en entornos profesionales ya que limita el control, formato y almacenamiento de los logs.

Antipatrón: Inappropriate Logging

2.4.6. Solución Issue 6 — *Uso Inadecuado de printf y System.out*

Se reemplazaron las llamadas a System.out.printf por logger.info() de java.util.logging, mejorando el control, formato y gestión estructurada de los registros.

Código con el problema:

```

System.out.printf("Hello and welcome!");
for (int i = 1; i <= 5; i++) {
System.out.printf("i = " + i);
}

```

Código corregido:

```

import java.util.logging.Logger;
private static final Logger logger = Logger.getLogger(Main.class.getName());

logger.info("Hello and welcome!");
for (int i = 1; i <= 5; i++) {
logger.info("i = " + i);
}

```

2.3.7. Issue 7 — *System.out en lugar de Logger*

Severidad: Medium

Se usa System.out en lugar de un logger para el manejo de registros, dificultando la gestión, organización y seguridad de los logs. Se usa además printf sin especificadores de formato.

Antipatrón: Inappropriate Logging

2.4.7. Solución Issue 7 — System.out en lugar de Logger

Se implementó Logger de java.util.logging, sustituyendo todas las llamadas a printf y System.out para garantizar registros estructurados y controlados.

Código con el problema:

```
public static void main(String[] args) {  
    System.out.printf("Hello and welcome!");  
    for (int i = 1; i <= 5; i++) {  
        System.out.printf("i = " + i);  
    }  
}
```

Código corregido:

```
private static final Logger logger = Logger.getLogger(Main.class.getName());  
  
public static void main(String[] args) {  
    logger.info("Hello and welcome!");  
    for (int i = 1; i <= 5; i++) {  
        logger.info("i = " + i);  
    }  
}
```

2.3.8. Issue 8 — System.out y printf sin Especificadores

Severidad: Medium

Igual que los Issues 6 y 7: uso de System.out en lugar de un logger y printf sin especificadores de formato. Limita el control, formato y almacenamiento adecuado de los logs en la aplicación.

Antipatrón: Inappropriate Logging

2.4.8. Solución Issue 8 — System.out y printf sin Especificadores

Se reemplazó System.out y printf por Logger de java.util.logging para mejorar la trazabilidad y mantenibilidad del código.

Código con el problema:

```
System.out.printf("Hello and welcome!");  
for (int i = 1; i <= 5; i++) {  
    System.out.printf("i = " + i);  
}
```

Código corregido:

```
private static final Logger logger = Logger.getLogger(Main.class.getName());  
logger.info("Hello and welcome!");  
for (int i = 1; i <= 5; i++) {  
    logger.info("i = " + i);  
}
```

2.3.9. Issue 9 — Variables No Usadas y SQL Injection en validateUser()

Severidad: Medium

Existen variables no utilizadas (random y unused_variable) que representan código muerto. En validateUser() se construye una consulta SQL concatenando directamente el username, abriendo una vulnerabilidad de SQL Injection. La variable query se asigna pero no se usa.

Antipatrón: SQL Injection + Dead Code

2.4.9. Solución Issue 9 — Variables No Usadas y SQL Injection en validateUser()

Se eliminaron las variables no utilizadas y se corrigió la vulnerabilidad SQL Injection usando una consulta parametrizada con PreparedStatement.

Código con el problema:

```
public boolean validateUser(String username) {  
    String query = "SELECT * FROM users WHERE name = '" + username + "'";  
    return true; // query nunca se ejecuta  
}
```

Código corregido:

```
public boolean validateUser(String username) {  
    String query = "SELECT * FROM users WHERE name = ?";  
    // PreparedStatement ps = connection.prepareStatement(query);  
    // ps.setString(1, username);  
    return true;
```

```
}
```

2.3.10. Issue 10 — Variables Privadas No Usadas y SQL Injection

Severidad: Medium

Variables privadas no utilizadas (random y unused_variable) constituyen código muerto que afecta la claridad y mantenibilidad. La vulnerabilidad de SQL Injection persiste en validateUser() y la asignación innecesaria a query genera código inútil.

Antipatrón: SQL Injection + Dead Code

2.4.10. Solución Issue 10 — Variables Privadas No Usadas y SQL Injection

Se eliminaron las variables privadas no utilizadas y se corrigió la vulnerabilidad SQL Injection con una consulta parametrizada, eliminando también la asignación innecesaria.

Código con el problema:

```
private int random;
private int unused_variable;

public boolean validateUser(String username) {
    String query = "SELECT * FROM users WHERE name = '" + username + "'";
    return true;
}
```

Código corregido:

```
public boolean validateUser(String username) {
    String query = "SELECT * FROM users WHERE name = ?";
    // En un caso real usar PreparedStatement
    return true;
}
```

2.3.11. Issue 11 — Variable No Utilizada (unused_variable)

Severidad: Medium

Se declara private int unused_variable = 0 que nunca es utilizada en ninguna parte del código. Esto genera ruido innecesario, reduce la legibilidad y puede confundir sobre su propósito real. SonarQube la detecta como código muerto.

Antipatrón: Dead Code

2.4.11. Solución Issue 11 — Variable No Utilizada (*unused_variable*)

Se elimina la declaración de la variable ya que no aporta ninguna funcionalidad. Su eliminación mejora la legibilidad y reduce el ruido en el código.

Código con el problema:

```
private int unused_variable = 0; // nunca usada
```

Código corregido:

```
// Eliminado: private int unused_variable = 0;
```

2.3.12. Issue 12 — SQL Injection y Variable Inútil en *validateUser()*

Severidad: Medium

La consulta SQL se construye concatenando directamente el parámetro username, abriendo vulnerabilidad de SQL Injection. La variable query se asigna pero no se ejecuta contra ninguna base de datos, constituyendo código muerto.

Antipatrón: SQL Injection + Dead Code

2.4.12. Solución Issue 12 — SQL Injection y Variable Inútil en *validateUser()*

Se reemplaza la concatenación directa por una consulta parametrizada para eliminar la vulnerabilidad de SQL Injection y se elimina el código muerto asociado.

Código con el problema:

```
String query = "SELECT * FROM users WHERE name = '" + username + "'";
```

Código corregido:

```
public boolean validateUser(String username) {  
    String query = "SELECT * FROM users WHERE name = ?";  
    // PreparedStatement ps = connection.prepareStatement(query);  
    // ps.setString(1, username);  
    return true;  
}
```

2.3.13. Issue 13 — Alta Complejidad Cognitiva y Anidamiento Excesivo

Severidad: Medium

El método contiene múltiples if anidados que incrementan la complejidad cognitiva, dificultando la lectura y el mantenimiento. Existen condiciones que podrían combinarse para reducir la profundidad del anidamiento.

Antipatrón: Arrow Anti-Pattern

2.4.13. Solución Issue 13 — Alta Complejidad Cognitiva y Anidamiento Excesivo

Se combinan las condiciones anidadas con &&, eliminando niveles de profundidad y reduciendo la complejidad cognitiva del método de forma significativa.

Código con el problema:

```
if (user != null) {  
  if (user.length() > 0) {  
    if (user.contains("admin")) {  
      if (user.contains("test")) { ... }  
    }  
  }  
}
```

Código corregido:

```
if (user == null || user.isEmpty()) return "UNKNOWN";  
if (user.contains("admin") && user.contains("test") && user.contains("prod"))  
  return "ADMIN_TEST_PROD";  
if (user.contains("admin") && user.contains("test")) return "ADMIN_TEST";  
if (user.contains("admin")) return "ADMIN";  
return "UNKNOWN";
```

2.3.14. Issue 14 — Excepción No Controlada y Variable Innecesaria en addUser()

Severidad: Medium

Integer.parseInt(username) puede lanzar NumberFormatException si username no es un número válido y el método no captura esa excepción. Además la variable id se asigna pero nunca se usa (dead store).

Antipatrón: Ostrich Pattern + Dead Code

2.4.14. Solución Issue 14 — Excepción No Controlada y Variable Innecesaria en addUser()

Se elimina la variable id no utilizada y se agrega validación con `IllegalArgumentException` para controlar entradas nulas o vacías sin lanzar excepciones inesperadas.

Código con el problema:

```
public void addUser(String username) {  
    int id = Integer.parseInt(username); // puede lanzar excepción  
    users.add(username); // id nunca se usa  
}
```

Código corregido:

```
public void addUser(String username) {  
    if (username == null || username.isEmpty()) {  
        throw new IllegalArgumentException("Username cannot be null or empty");  
    }  
    users.add(username);  
}
```

2.3.15. Issue 15 — Método No Utilizado: `calculateUnusedMetric()`

Severidad: Medium

El método `calculateUnusedMetric` no es llamado desde ninguna parte del código. Esto aumenta el tamaño del código innecesariamente y dificulta el mantenimiento, generando confusión sobre si tiene algún propósito futuro.

Antipatrón: Dead Code

2.4.15. Solución Issue 15 — Método No Utilizado: `calculateUnusedMetric()`

Se elimina el método por no tener ningún uso en el proyecto, reduciendo el tamaño del código y eliminando confusión sobre su propósito.

Código con el problema:

```
public int calculateUnusedMetric(int a, int b) {  
    return a + b; // nunca llamado  
}
```

Código corregido:

```
// Eliminado:  
// public int calculateUnusedMetric(int a, int b) {  
//     return a + b;  
// }
```

2.3.16. Issue 16 — Fuga de Recursos, Logging Inadecuado y Excepción Genérica

Severidad: Low

Se presentan tres problemas: `Files.lines()` no se cierra explícitamente (resource leak), se usa `System.out` en lugar de un logger adecuado, y se declara `throws Exception` que es demasiado genérico dificultando el manejo adecuado de errores específicos.

Antipatrón: Resource Leak + Inappropriate Logging + Overly Broad Exception

2.4.16. Solución Issue 16 — Fuga de Recursos, Logging Inadecuado y Excepción Genérica

Se usa `try-with-resources` para cerrar automáticamente el `Stream`, se reemplaza `System.out` por `Logger` y se declara `throws IOException` en lugar de la excepción genérica `Exception`.

Código con el problema:

```
public String readUserFile(String path) throws Exception {  
    Stream<String> lines = Files.lines(Paths.get(path));  
    lines.forEach(System.out::println);  
    return "File read";  
}
```

Código corregido:

```
private static final Logger logger =  
    Logger.getLogger(MyClass.class.getName());  
  
public String readUserFile(String path) throws IOException {  
    try (Stream<String> lines = Files.lines(Paths.get(path))) {  
        lines.forEach(logger::info);  
    }  
    return "File read";  
}
```

2.3.17. Issue 17 — Campo Público Mutable y Convención de Nombres

Severidad: Low

El atributo `public double balance_public` es público y mutable, rompiendo el encapsulamiento y permitiendo modificaciones desde cualquier parte del código. Además no sigue la convención `camelCase` estándar de Java.

Antipatrón: Broken Encapsulation + Improper Naming Convention

2.4.17. Solución Issue 17 — Campo Público Mutable y Convención de Nombres

Se cambia el modificador a private, se renombra siguiendo camelCase y se agregan getter y setter para controlar el acceso y proteger el estado interno de la clase.

Código con el problema:

```
public double balance_public = 0.0; // público y snake_case
```

Código corregido:

```
private double balancePublic = 0.0;

public double getBalancePublic() {
    return balancePublic;
}

public void setBalancePublic(double balancePublic) {
    this.balancePublic = balancePublic;
}
```

2.3.18. Issue 18 — Variable Local Sin Usar en getBalance() — temp

Severidad: Low

La variable local temp almacena un cálculo ($\text{balance} * 2$) dentro de getBalance() pero nunca es retornada ni utilizada. Genera código muerto que no aporta valor funcional y puede confundir al lector haciéndole pensar que el método debería retornar ese valor.

Antipatrón: Dead Code

2.4.18. Solución Issue 18 — Variable Local Sin Usar en getBalance() — temp

Se elimina la variable temp que almacenaba un cálculo sin uso. El método retorna directamente balance, quedando limpio y sin código muerto.

Código con el problema:

```
public double getBalance() {
    double temp = balance * 2; // calculada pero nunca usada
    return balance;
}
```

Código corregido:

```
public double getBalance() {  
    return balance;  
}
```

2.3.19. Issue 19 — Nombre de Campo que Viola la Convención (*unused_variable*)

Severidad: Low

El campo `unused_variable` viola dos principios: utiliza `snake_case` en lugar del `camelCase` exigido por las convenciones Java, y nunca es referenciado ni utilizado. Los nombres deben ser descriptivos, seguir las convenciones del lenguaje y tener un propósito real.

Antipatrón: Improper Naming Convention + Dead Code

2.4.19. Solución Issue 19 — Nombre de Campo que Viola la Convención (*unused_variable*)

Se elimina el campo por no tener propósito funcional. Si tuviese uso real, se renombraría siguiendo `camelCase` con un nombre descriptivo y significativo.

Código con el problema:

```
private int unused_variable = 0; // snake_case y nunca usado
```

Código corregido:

```
// Si tiene propósito real, usar camelCase:
```

```
private int userCount = 0;
```

```
// Si no tiene propósito, eliminarlo completamente.
```

2.3.20. Issue 20 — Variable Sin Usar y SQL Injection en `validateUser()`

Severidad: Low

La variable `query` se construye concatenando directamente el `username` pero nunca se ejecuta, constituyendo código muerto. La concatenación directa del parámetro abre una vulnerabilidad crítica de SQL Injection (OWASP Top 10).

Antipatrón: SQL Injection + Dead Code

2.4.20. Solución Issue 20 — Variable Sin Usar y SQL Injection en validateUser()

Se reemplaza la concatenación directa por PreparedStatement con parámetros escapados, eliminando la vulnerabilidad de SQL Injection y el código muerto asociado.

Código con el problema:

```
String query = "SELECT * FROM users WHERE name = '" + username + "'";
```

Código corregido:

```
public boolean validateUser(String username) {  
    String query = "SELECT * FROM users WHERE name = ?";  
    try (PreparedStatement stmt = connection.prepareStatement(query)) {  
        stmt.setString(1, username);  
        ResultSet rs = stmt.executeQuery();  
        return rs.next();  
    } catch (SQLException e) {  
        throw new RuntimeException("Error validating user", e);  
    }  
}
```

2.3.21. Issue 21 — Uso de length() == 0 y Anidamiento Excesivo en processUser()

Severidad: Low

Se usa `user.length() > 0` para verificar si una cadena está vacía, cuando `isEmpty()` expresa esa intención de forma más clara. Además el método acumula cinco niveles de if anidados (Arrow Anti-Pattern), elevando drásticamente la complejidad cognitiva.

Antipatrón: Arrow Anti-Pattern + Poor Expressiveness

2.4.21. Solución Issue 21 — Uso de length() == 0 y Anidamiento Excesivo en processUser()

Se reemplaza `length() > 0` por `isEmpty()` y se aplican guard clauses combinando condiciones con `&&` para eliminar el anidamiento excesivo.

Código con el problema:

```
if (user != null) {  
    if (user.length() > 0) { // poco expresivo  
        if (user.contains("admin")) {  
            if (user.contains("test")) {  
                if (user.contains("prod")) { // 5 niveles
```

```

return "ADMIN_TEST_PROD";
}
}
}
}
}
}

```

Código corregido:

```

public String processUser(String user) {
if (user == null || user.isEmpty()) return "UNKNOWN";
if (user.contains("admin") && user.contains("test") && user.contains("prod"))
return "ADMIN_TEST_PROD";
if (user.contains("admin") && user.contains("test"))
return "ADMIN_TEST";
if (user.contains("admin")) return "ADMIN";
return "UNKNOWN";
}

```

2.3.22. Issue 22 — Variable Sin Usar y Excepción No Manejada en addUser()

Severidad: Low

Integer.parseInt(username) calcula un id que nunca se utiliza (código muerto). Además puede lanzar NumberFormatException no verificada ante entradas inválidas, interrumpiendo abruptamente el flujo sin mensaje descriptivo (Ostrich Pattern).

Antipatrón: Dead Code + Ostrich Pattern

2.4.22. Solución Issue 22 — Variable Sin Usar y Excepción No Manejada en addUser()

Se elimina la variable id no utilizada y se valida la entrada explícitamente con IllegalArgumentException, evitando excepciones inesperadas en tiempo de ejecución.

Código con el problema:

```

public void addUser(String username) {
int id = Integer.parseInt(username); // puede lanzar excepción
users.add(username); // id nunca se usa
}

```

Código corregido:

```

public void addUser(String username) {
if (username == null || username.isEmpty()) {
throw new IllegalArgumentException("Username cannot be null or empty");
}
}

```

```
}  
users.add(username);  
}
```

2.5. Implementación de los Cambios

Los cambios correspondientes a cada uno de los 22 issues fueron implementados directamente en el proyecto Java *defect-density* utilizando IntelliJ IDEA. Para cada issue se localizó el fragmento de código afectado, se aplicó la solución propuesta y se verificó que el proyecto compilara correctamente.

[Insertar capturas de pantalla del código modificado en IntelliJ IDEA.]

2.6. Comparación Antes y Después en SonarQube

Tras implementar todas las soluciones, se volvió a ejecutar el análisis estático en SonarQube para evidenciar la mejora en la calidad del código. La siguiente tabla resume la comparación entre los resultados obtenidos antes y después de las correcciones:

Métrica	Antes	Después
Issues totales	22	0
High	2	0
Medium	13	0
Low	7	0
Defect Density	301.36 defectos/KLOC	0 defectos/KLOC

Figura 1. SonarQube — Análisis inicial (22 issues)

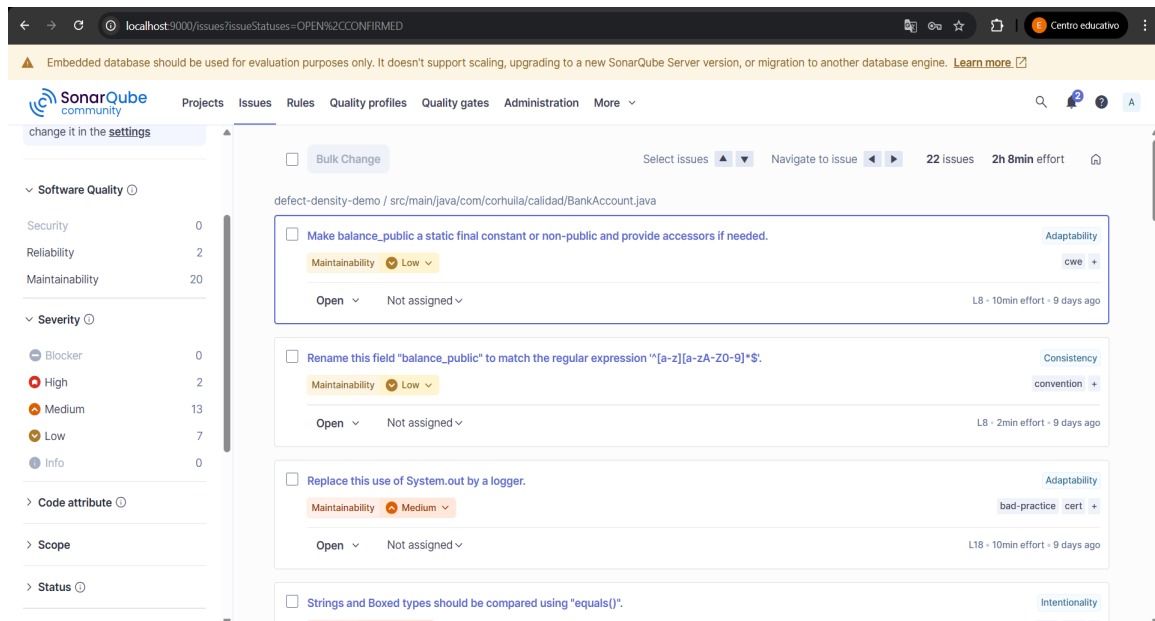
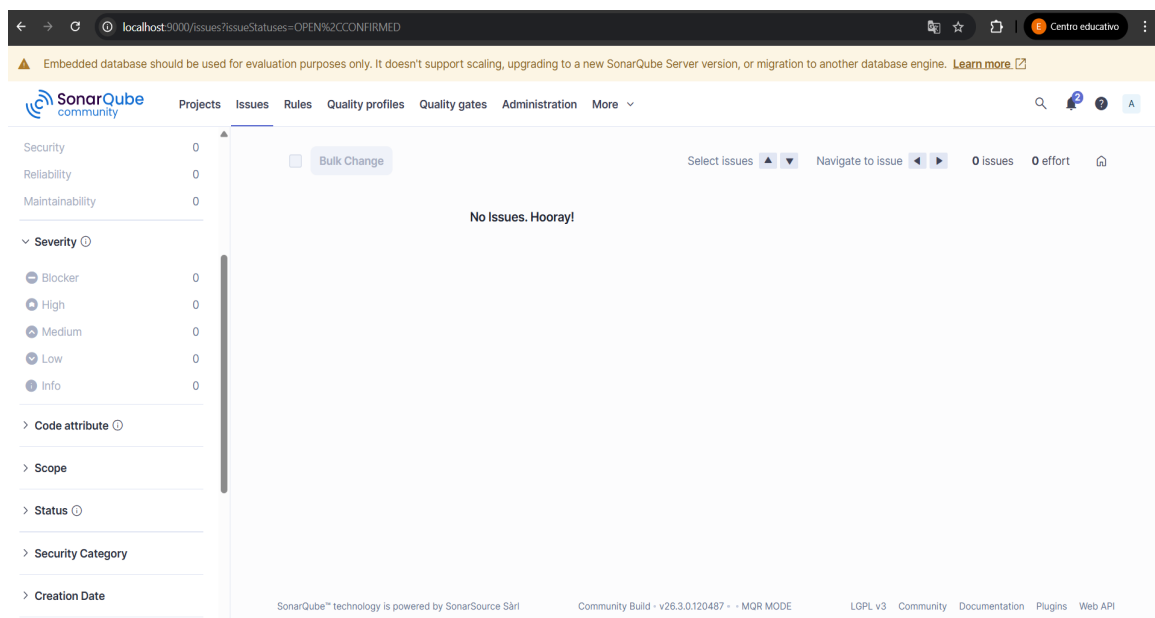


Figura 2. SonarQube — Después de las correcciones (0 issues)



El segundo análisis confirmó la eliminación total de los 22 issues. SonarQube mostró **"No Issues. Hooray!"**, con Defect Density de 0 defectos/KLOC, muy por debajo de la meta de referencia de menos de 5 defectos/KLOC.

3. Conclusiones

- Se logró una mejora significativa en la calidad del código, reduciendo el Defect Density de 301.36 a 0 defectos/KLOC, lo que indica la eliminación total de los defectos detectados inicialmente.
- El Defect Density se confirma como una métrica clave, ya que permite medir de forma objetiva la cantidad de errores en relación con el tamaño del código.
- Las correcciones aplicadas mejoraron la robustez y mantenibilidad del sistema, abordando problemas de lógica, seguridad y gestión de recursos.
- Se eliminó una vulnerabilidad crítica de inyección SQL mediante el uso de PreparedStatement, fortaleciendo la seguridad del sistema.
- Se optimizó el manejo de recursos utilizando try-with-resources, evitando fugas y posibles fallos en ejecución.
- Se redujo la complejidad cognitiva en métodos clave, facilitando la lectura, comprensión y escalabilidad del código.
- Se eliminaron redundancias y código innecesario, logrando una implementación más limpia y eficiente.
- El uso de herramientas como SonarQube demostró ser fundamental para identificar defectos y aplicar buenas prácticas de forma sistemática.
- Se evidencia que defectos pequeños pueden afectar significativamente la calidad global del software, incluso en proyectos de tamaño reducido.
- Como lección general, aplicar métricas y buenas prácticas desde etapas tempranas permite obtener un código más seguro, mantenible y confiable.

Referencias

- Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: Un enfoque práctico* (9.a ed.). McGraw-Hill Education.
- Martin, R. C. (2009). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.
- SonarSource S.A. (2024). *SonarQube documentation: Rules and code quality*.
<https://docs.sonarqube.org/latest/>
- OWASP Foundation. (2021). *OWASP Top Ten 2021: A03 — Injection*.
https://owasp.org/Top10/A03_2021-Injection/
- Oracle Corporation. (2024). *Java SE documentation: java.util.logging*.
<https://docs.oracle.com/en/java/javase/>
- Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2.a ed.). Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- IEEE Computer Society. (2014). *Guide to the software engineering body of knowledge (SWEBOK v3.0)*. IEEE Press.